Group 01

Chris Cornelson

Erick Granados

Jimmy Tun

Week 3 – Sequential Solution

Updated Problem Statement

For this week, Group 01 focused on building a simple sequential CPU version of the grayscale image converter before moving into CUDA. The goal was to verify correctness first. The grayscale value was computed using the formula:

$$Gray = 0.299R + 0.587G + 0.114B$$

Group 01 chose this formula because it is standard, simple, and easy to check by hand with small test cases.

Sample Input / Output Data

Group 01 tested the program using a small set of RGB pixels:

- (255, 0, 0)
- (0, 255, 0)
- (0, 0, 255)
- (255, 255, 255)
- (0, 0, 0)

Expected grayscale outputs:

- 76
- 149
- 29
- 255
- 0

These values were chosen because they are easy to verify manually.

Sequential CPU Approach

The CPU version processes one pixel at a time using a simple loop. For each pixel, the program reads the RGB values, applies the grayscale formula, and stores the grayscale result. Group 01 chose this approach because it is the most basic version of the solution and gives a clear baseline before parallelization.

Sample Test Output

The CPU program produced the following output:

- Pixel 0 -> 76
- Pixel 1 -> 149
- Pixel 2 -> 29
- Pixel 3 -> 255
- Pixel 4 -> 0

The output matched the expected grayscale values, which confirmed that the sequential logic was working correctly.

Sequential CPU Approach (Explanation)

For the CPU version, we process one pixel at a time using a simple loop.

For each pixel:

1. Read the R, G, and B values
2. Apply the grayscale formula
3. Store the result

We chose this approach because it is the most basic and easiest to understand. This works well because it gives us a clear baseline that we can compare against later when we implement the GPU version.

```
CPU Implementation (C++ Code)
// File: cpu_grayscale.cpp
// Version: 1.0
// Description:
//   This is our basic CPU version of grayscale conversion.
//   We made this first so we can verify correctness before moving to CUDA.

#include <iostream>
#include <vector>
#include <chrono>

using namespace std;

// We are using this struct to group RGB values together for each pixel.
struct Pixel {
    int r;
    int g;
    int b;
};

// We created this function to convert one pixel to grayscale.
int convertToGray(const Pixel& p) {

    // We use the standard grayscale formula from our problem statement
    // because it is simple and widely used.
    double gray = 0.299 * p.r + 0.587 * p.g + 0.114 * p.b;
```

```
    // Here we convert to int because grayscale pixel values are whole numbers
    return (int)gray;
}

int main() {

    // We created a small test dataset so that we can easily verify correctness
    vector<Pixel> pixels = {
        {255, 0, 0},
        {0, 255, 0},
        {0, 0, 255},
        {255, 255, 255},
        {0, 0, 0}
    };

    vector<int> grayValues(pixels.size());

    // We start the timing before the loop begins
    auto start = chrono::high_resolution_clock::now();

    // This is where we process each pixel one at a time (sequential execution)
    for (int i = 0; i < pixels.size(); i++) {
        grayValues[i] = convertToGray(pixels[i]);
    }

    // Here is where we stop timing after the loop finishes
    auto end = chrono::high_resolution_clock::now();

    chrono::duration<double, milli> duration = end - start;

    // This is where the program prints results so the team can verify correctness
    cout << "Grayscale Results:" << endl;
    for (int i = 0; i < pixels.size(); i++) {
        cout << "Pixel " << i << " -> " << grayValues[i] << endl;
    }

    // This is where the program prints execution time for the CPU version
    cout << "CPU Execution Time: " << duration.count() << " ms" << endl;

    return 0;
}
```

Sample Test Output (Description)

When we ran the program, it printed grayscale values for each pixel and confirmed that the results matched our expected values.

This helped us confirm that the grayscale formula is implemented correctly in the CPU version.

CPU Timing

Group 01 measured CPU execution time using `std::chrono::high_resolution_clock`. The measured runtime was:

```
0.00022 ms
```

This time was very small because the test case only contained 5 pixels. Group 01 chose to begin with a very small dataset so correctness could be verified clearly before moving to larger data and CUDA later.

As shown in Figure 1, Chris compiled the program and then ran it, and the program produced the expected results.
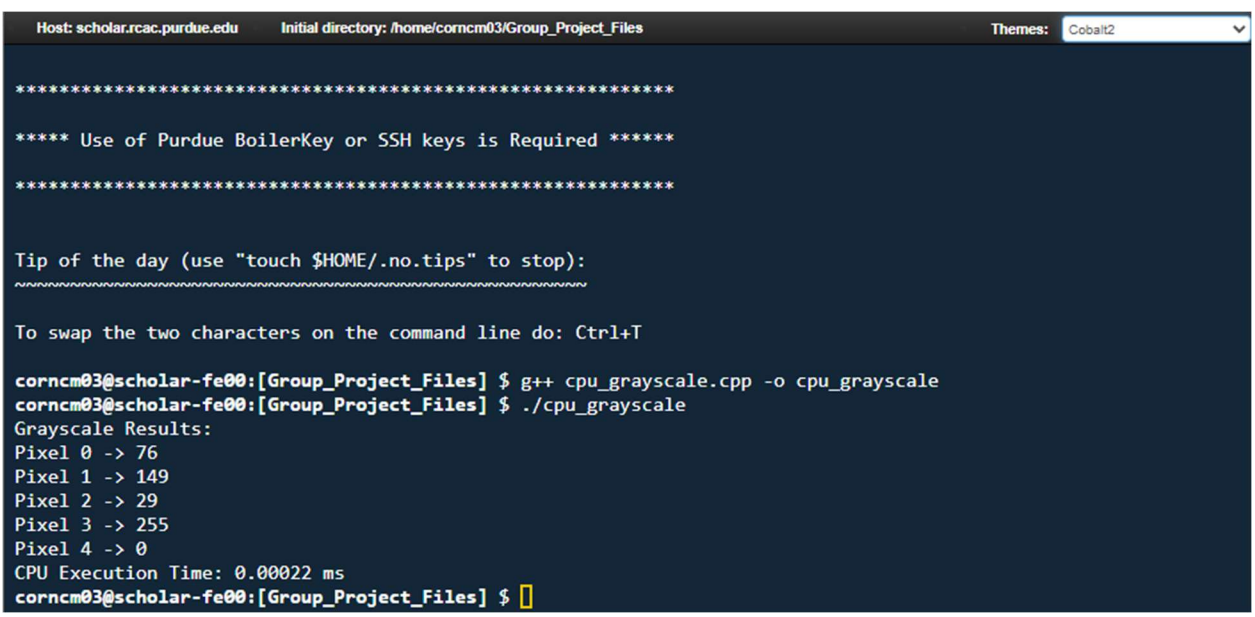
Figure 1